DATAMORPHO

Whitepaper

A Publication-Ready Technical Release for Datamorpho v0.001

Author: Germán Abal (g@evvm.org)

Reviewers:
- TBA
- TBA
- TBA

Version: 0.001
Status: Public Draft
Document Type: Whitepaper
Companion Document: *Datamorpho Specification v0.001 (Normative Core Specification)*
Reference Software License Intention: MIT
Source and Discussion: github.com/ariutokintumi/datamorpho
Announcements: x.com/datamorpho
Website: datamorpho.io

A File Standard for Multi-State Concealed Content

Datamorpho is an open file standard and protocol model for creating files that remain valid in their original format while containing one or more concealed alternate states. A Datamorphed file explicitly declares that hidden states exist and where reconstruction information may later be found, while withholding the structural and secret material needed to reconstruct those hidden states.

This whitepaper explains:

- what Datamorpho is,
- why it exists,
- how it works,
- what security properties it aims to provide,
- what it does and does not claim,
- and why it deserves open technical discussion and standardization work.

Initial carrier profiles in v0.001: JPEG, TXT, PDF
Next immediate media targets after first release: Audio, Video
Project direction: Open specification, open reference software, future public review, eventual RFC-track discussion

---

Changelog

v0.001 — Public Draft Whitepaper

- First formal public whitepaper release aligned with the normative Datamorpho Specification v0.001
- Defines Datamorpho as a **multi-state concealed-content file standard**
- Clarifies that Datamorpho is **not steganography**
- Formalizes the separation between:
  - public declaration,
  - concealed payload,
  - and state-specific reconstruction objects
- Establishes three initial carrier profiles:
  - JPEG
  - TXT
  - PDF
- Establishes layout strategies:
  - `sparse`
  - `sparse-with-chaff`
- Clarifies that:
  - states do **not** need to be sequential,
  - hidden states may be treated as an unordered set,
  - and fragments of the same state may use different cryptographic suites
- Adopts the v0.001 position that reconstruction objects are **complete secret-bearing artifacts**, including key material
- Adds digest cross-binding between public state descriptors and reconstruction objects
- Adds MorphoStorage as a generalized locator model, not merely a URL list
- Expands use cases to include:
  - NFTs
  - games
  - media releases
  - financial order books
  - auctions
  - news releases
  - censorship-resistant wide distribution with later key release
  - future audio and video distribution

- Frames the standard as open, lawful, and intended for constructive use only

Historical Origin

Datamorpho's public origin traces back to **ETHGlobal Mexico 2022**, where it was showcased as a **finalist project** with an early focus on dynamic files and NFT pre-reveal / reveal logic to avoid scams: https://ethglobal.com/showcase/datamorpho-omvp5. It was judged by Stani Kulechov representing Lens/AAVE and Chuy García representing the hackathon organization.

This publication is the first formal open specification-era release, not a second normative version

---

Acknowledgements

Datamorpho's early public prototype work involved a hackathon team context in addition to the primary author and standard designer:

- Ben Dumoulin (x.com/BenDumoulin) - early PoC implementation
- R. Benson Evans (x.com/iglobecreator) - early PoC research
- Eduardo Davalos (x.com/metaversearchi_) - early PoC design

These acknowledgements are intended to distinguish **early prototype contributors** from the formal authorship of the present specification and whitepaper.

---

1. Abstract

Datamorpho is a file standard and protocol model for creating files that preserve an ordinary visible representation while also containing one or more concealed alternate states.

Datamorpho is **not steganography**. In steganographic systems, the existence of hidden content is typically meant to remain undisclosed or deniable. Datamorpho explicitly requires the opposite: a Datamorphed file is expected to declare that hidden states exist and to provide enough public direction for reconstruction information to later be located properly.

Datamorpho is therefore not a "hidden-message" system in the classical sense. It is a **declared latent-state file architecture**.

In Datamorpho v0.001, a file is divided conceptually into:

- a valid base carrier,
- a public manifest,

- one or more concealed payload regions,
- and one or more state-specific reconstruction objects.

A reconstruction object identifies exactly one hidden state and may contain:

- fragment locations,
- fragment ordering,
- cryptographic suite assignments,
- layout strategy,
- digest bindings,
- and the key material required to reconstruct that state.

Datamorpho aims to raise the cost of unwanted disclosure by combining:

- sound cryptography,
- sparse placement,
- optional chaff,
- non-monotonic fragment ordering,
- state-specific reconstruction objects,
- and public/private layer separation.

This whitepaper describes Datamorpho as a serious technical system, not a mystical one. It does not claim impossibility. It claims **layered resistance**.

---

2. Executive Summary

Datamorpho proposes a new way to think about files.

A file does not need to be limited to one visible meaning. It can be a valid ordinary JPEG, TXT, or PDF file and still contain one or more hidden states that become reconstructable only when the appropriate reconstruction object is obtained.

The key innovation is not simply that bytes are hidden. The innovation is that the file can:

- publicly declare that hidden states exist,
- publicly describe the expected triggers or reveal conditions,
- publicly indicate where reconstruction information may later be found,
- and still keep the actual recovery of those states dependent on a separate secret-bearing reconstruction object.

A hidden state in Datamorpho does not need to be stored as one contiguous payload. It may be assembled from:

- multiple fragments,
- physically disordered offsets,
- sparse placement,
- payload regions shared with other states,
- and optional chaff.

The state may also use different cryptographic suites across different fragments of the same state.

This gives Datamorpho a distinctive profile:

- **not steganography**
- **not ordinary encryption**
- **not just a viewer trick**
- **not just a metadata swap mechanism**
- but a real latent-state file standard.

---

3. What Datamorpho Is - and Is Not

3.1 What Datamorpho Is

Datamorpho is:

- a file standard proposal,
- a carrier-aware concealment protocol,
- a public/private layered content model,
- a state-specific reconstruction architecture,
- and a foundation for future interoperable file creation and interpretation systems.

3.2 What Datamorpho Is Not

Datamorpho is **not steganography**.

In classical steganography, the hidden content is not supposed to be publicly declared. Datamorpho requires the reverse: a Datamorphed file openly declares that hidden states exist and points toward how reconstruction information may later be located.

Datamorpho is also not:

- merely an encrypted trailer,
- merely a hidden file inside another file,
- or merely a reveal trick for one NFT use case.

It is a structured, declared, multi-state file model.

---

4. Motivation and Problem Statement

Traditional files are static. Traditional encrypted files are usually monolithic. Traditional reveal systems often require moving or changing public metadata. Datamorpho addresses a different class of problems:

How can a file remain ordinary and valid, openly declare that additional states exist, and still make those states hard to recover without the correct reconstruction object?

That matters because many real systems need controlled latent content.

Examples include:

- NFT pre-reveal / post-reveal without changing the metadata location
- game items and unlockable assets
- staged media and art releases, music global launch
- archival or institutional timed disclosure
- financial order books in which an order exists but the identity behind it is only disclosed later
- auction systems where hidden bid-related or participant-related state becomes visible only at the correct moment
- news releases that are widely distributed before publication but only become readable when the proper release artifact is made available
- censorship-resistant lawful publication where a file can spread widely first and reconstruction material can be released later
- collectible files whose hidden state depends on ownership, minting, or later release
- free preview - paid content gate

Datamorpho therefore solves a general problem of **declared latent content**.

---

5. Foundational Premises

Datamorpho rests on the following premises.

5.1 A file can be more than its visible state

The visible file is only one interpretation of the full object.

5.2 Public declaration and private reconstruction should be separate

The public file should reveal enough for discoverability and interoperability, but not enough for recovery.

5.3 Security is layered

Cryptography protects meaning. Sparse placement, chaff, and disorder raise attack cost. Reconstruction secrecy protects the recovery map.

5.4 States do not need to be sequential

Hidden states may be released in any order or no order at all. Datamorpho should be understood as supporting a **set of states**, not requiring a linear sequence.

5.5 Reconstruction should be state-specific

A reconstruction object should identify exactly one hidden state, allowing independent release and compartmentalized disclosure.

5.6 The standard should remain agnostic about viewers and decryption workflows

Datamorpho should standardize the file structure, public declaration model, payload semantics, and reconstruction semantics. It should **not** attempt to standardize every possible viewer, decryption UX, trigger-validation engine, or future interpretation model. Those will vary across implementations and evolve over time.

That is the correct boundary for the standard.

---

6. Core Architecture

A Datamorphed file consists conceptually of four layers.

6.1 Base Carrier

The ordinary visible file.

6.2 Public Manifest

The public declaration that says:

- this file is Datamorpho,
- this is the Datamorpho version,
- these states exist,
- these are the declared triggers,
- these are the MorphoStorage directions,
- and these are the expected reconstruction-object digests.

6.3 Concealed Payload

One or more payload regions containing the concealed bytes from which hidden states may be reconstructed.

6.4 Reconstruction Object

A state-specific secret-bearing artifact that includes:

- the target state identifier,
- fragment map,
- offset and length information,
- logical reassembly order,
- cryptographic suite instructions,
- layout strategy,
- digest bindings,
- and, in v0.001, the key material necessary to reconstruct the target state.

---

7. Why Reconstruction Objects Are Complete Secret-Bearing Artifacts in v0.001

Datamorpho v0.001 takes a direct and pragmatic position:

If the reconstruction object is already secret-critical because it reveals the fragment map, ordering, layout, and suite instructions, then there is no strong architectural reason to forbid it from also carrying the key material required to reconstruct the target hidden state.

That choice is intentional.

In other words, Datamorpho v0.001 treats the reconstruction object as a **complete combination box** for one hidden state.

This has several advantages:

- it simplifies the first standard release,
- it makes implementations easier to test,
- it avoids pretending that the map is “less sensitive” than the key,
- it keeps the core standard simple,
- and it leaves deployment-specific key separation for future profiles or implementations that truly need it.

Could some deployments still choose to separate key material later? Yes. There may be operational reasons:

- different trust domains,
- cheaper ultra-secret storage of a small key versus a full JSON object,
- onchain or hardware-assisted release models,
- threshold release,
- FHE-assisted or TEE-assisted key handling,
- or legal and organizational segregation of duties.

But those are deployment choices, not a reason to weaken the first core model.

So the v0.001 position is clear:

the reconstruction object is the secret object.

---

8. Security Model

Datamorpho’s security does not come from a single secret trick. It comes from composition.

8.1 Cryptography protects content meaning

A fragment recovered from a payload region is still not useful unless its cryptographic protection can be correctly interpreted and the required key material is available.

Datamorpho does not mandate one fixed algorithm family, but it is designed to accommodate both conventional and post-quantum-oriented deployments. NIST finalized ML-KEM in FIPS 203 and ML-DSA in FIPS 204 in August 2024, and explicitly positions those standards as the foundation for post-quantum key establishment and signatures going forward.

8.2 Sparse placement breaks monolithic assumptions

A hidden state does not need to exist as one continuous payload.

8.3 Chaff raises ambiguity

In `sparse-with-chaff`, bytes not used by the target state are intentionally present to increase ambiguity and attack cost.

8.4 Logical order is independent from physical order

Offsets can be physically unordered while the reconstruction order remains fully deterministic.

8.5 Heterogeneous cryptography within one state increases flexibility

One state may use multiple cryptographic suites across different fragment spans.

8.6 Public declaration without recovery leakage is strategically valuable

The file can openly say “there is more here” while still withholding the actual path to reconstruction.

---

9. What Datamorpho Does Not Claim

Datamorpho does **not** claim:

- impossibility of compromise,
- magical security from obscurity,
- that sparse layout replaces cryptography,
- or that future cryptographic change is unnecessary.

If a reconstruction object is exposed, that exposure is potentially serious precisely because the reconstruction object is a secret-bearing artifact in v0.001.

The standard is strong when it is honest about that.

---

10. Cryptographic Agility

Datamorpho does not mandate one universal algorithm family.

It allows:

- one suite for one fragment,
- another suite for another fragment,
- a shared state-level key with fragment-level overrides,
- and different hidden states using different cryptographic mixes.

This is one of Datamorpho's most important properties. It avoids locking the standard into one cryptographic era or one rigid implementation style.

The standard is therefore about:

- how a state is declared,
- how a state is embedded,
- how a state is reconstructed,
- and how those instructions are represented,

not about forcing one cryptographic monoculture.

For browser-oriented implementations, the Web Crypto API and `SubtleCrypto` provide a practical baseline for low-level cryptographic operations and are widely available across modern browsers. That matters because Datamorpho is explicitly designed with future browser extensions and JS-based interpreters in mind.

---

11. Layout Strategy

Datamorpho v0.001 standardizes two layout strategies.

11.1 Sparse

Meaningful bytes are non-contiguous and only the referenced spans matter.

This option remains important for practical reasons. In some deployments, especially with very large files, `sparse` may be preferred because it reduces payload expansion, storage cost, transfer size, and creation overhead, while still preserving non-contiguous placement and non-monotonic reconstruction. In other words, `sparse` exists because there are real cases where the additional size and bandwidth cost of chaff is not worth paying, even though sparse layout alone still provides structural resistance.

11.2 Sparse-with-Chaff

Meaningful bytes are non-contiguous and the unreferenced bytes are intentionally present as ambiguity-increasing material.

Those unreferenced bytes may include:

- chaff,
- arbitrary filler,
- bytes belonging to other states,
- or combinations of the above.

This means a Datamorpho payload is not best understood as “one hidden blob.” It is better understood as a **field of bytes in which only the reconstruction object tells you which spans matter for which state**.

---

12. Carrier Profiles

12.1 JPEG

The JPEG profile preserves the visible JPEG through the normal end-of-image marker and then appends the Datamorpho Binary Block afterward.

12.2 TXT

The TXT profile appends a terminal Datamorpho envelope after the visible text.

12.3 PDF

The PDF profile appends Datamorpho objects and payload streams in a PDF incremental-update section.

These three profiles were chosen because they represent:

- media,
- text / metadata,
- and structured documents.

That is a strong initial foundation.

---

13. PDF as a High-Value Carrier

PDF is particularly important because it is widely used for:

- formal publications,
- certificates,
- submissions,
- institutional records,
- legal and policy documents,
- e-books, free-preview to pay-to-read content,
- news, announcements, invitations,
- and collectible document objects.

Datamorpho PDF can represent not only hidden fragments, but full alternate hidden PDF states. That makes it especially suitable for staged disclosure and document-based reveal systems.

---

14. Games, Items, and Interactive Assets

Games are one of the strongest use cases for Datamorpho.

A Datamorphed file can represent:

- a hidden item,
- a location,
- a premium asset,
- a quest reward,
- a lore object,
- a puzzle clue,
- or an ownership-dependent interactive object.

This is especially useful because a game state often needs latent content that becomes available only when a condition is met. Datamorpho gives that logic a file-level architecture.

---

15. NFTs and Static Metadata Reveal

One of Datamorpho's earliest motivating use cases was the NFT pre-reveal / post-reveal problem.

Instead of changing the metadata location later, what can lead to scams and targeted distribution, Datamorpho allows a more elegant model:

- the file is already complete,
- the hidden state is already embedded,
- the metadata location can remain static,
- and the reveal happens through release of the reconstruction object and secret material.

This idea was already present in the early public Datamorpho prototype showcased at ETHGlobal Mexico 2022, where the project description explicitly highlighted trusted NFT mints with pre-reveal / reveal states as an immediate use case.

---

16. Financial, Auction, News, and Wide-Distribution Use Cases

Datamorpho also fits several high-value information-release scenarios.

16.1 Financial Order Books

An order can be distributed or committed while the identity behind the order remains concealed until a later reveal condition.

16.2 Auctions

Auction-related data can be distributed in latent form and only become reconstructable at the correct moment.

16.3 News Releases

A news file can be broadly distributed before publication and only later become reconstructable.

16.4 Censorship-Resistant Wide Distribution

A message file can spread first, while the secret-bearing reconstruction object is released later.

These are not all the same use case. But they all share the same need: **early distribution, later reconstruction**.

---

17. MorphoStorage as a Locator Layer

MorphoStorage answers a practical question:

Where should someone look for the reconstruction object of this state?

That answer may be simple or complex.

Simple examples:

- IPFS plus a **Content Identifier (CID)** only
- a URI to a JSON document
- a direct textual instruction

More complex examples:

- a smart contract address plus chain ID plus function and parameters
- an implementation-defined retrieval path
- a hybrid release system

MorphoStorage is therefore not one storage network. It is a generalized locator and direction layer.

---

18. Digest Cross-Binding

Datamorpho v0.001 uses cross-binding to reduce ambiguity.

Each state descriptor in the public manifest includes the digest of the intended reconstruction object.

Each reconstruction object includes the digest of the carrier file it belongs to.

This means:

- the public layer can point to the right reconstruction artifact,
- and the reconstruction artifact can confirm the right carrier file.

That is a major structural strength of the standard.

---

19. Threat Model

Datamorpho is meant to increase the cost of attack against:

- casual readers,
- ordinary file tools,
- naïve scanners,
- automated carving attempts,
- partial analysts,
- and many structured investigations that do not possess the correct reconstruction object.

It is not a cure for:

- broken endpoints,
- exposed secret-bearing reconstruction objects,
- weak operational security,
- weak cryptographic choices,
- or careless implementation.

Datamorpho is a resistance architecture, not a magical exemption from security engineering.

---

20. Responsible Use

Datamorpho is intended for lawful, ethical, and constructive use.

Appropriate uses include:

- publishing,
- archives,
- games,
- collectibles,
- staged media releases,
- educational assets,
- research,

- institutional disclosure workflows,
- and censorship-resistant lawful publication models.

It is not designed, promoted, or endorsed for criminal concealment, fraud, extortion, abuse, or illegal conduct.

---

21. Why Datamorpho Deserves Formal Open Discussion

Datamorpho is substantial enough to merit open technical review because it defines:

- a file capability,
- a carrier model,
- a public/private state model,
- a reconstruction-object model,
- a layout model,
- and an extensibility path.

It is exactly the kind of system that benefits from:

- a public specification,
- open-source reference code,
- sample files,
- interoperability vectors,
- and later standardization discussion.

RFC documents are the core output of the IETF, and not all RFCs are Internet Standards; Informational and Experimental publication tracks are explicitly part of the system and are a natural fit for new technical models like this in their early public life.

---

22. Why Open Reference Software Matters

A standard without software remains abstract.

An MIT-licensed reference implementation matters because it will:

- prove the spec is implementable,

- produce sample files,
- support debugging,
- enable review,
- and build trust in the format.

The first software phase does not need to solve every viewer or every release mechanism. It needs to create correct Datamorphed files and correct reconstruction objects.

That is enough to establish the foundation.

---

23. Immediate Next Targets: Audio and Video

After JPEG, TXT, and PDF, the next immediate targets are naturally **audio and video**.

This is not only because they are popular media types. It is because they express one of Datamorpho's strongest value propositions:

distribute first, reconstruct later.

Examples include:

- a film distributed before synchronized premiere time, with reconstruction released only at the correct moment
- a song or album distributed in latent form and unlocked later
- licensed media that becomes reconstructable only after rights payment
- large-scale coordinated releases where everyone receives the file in advance but nobody can reconstruct the hidden state until the release artifact appears

This is technically attractive and psychologically powerful. A user may possess the carrier file while still lacking the ability to recover the hidden state. That creates a strong sense of latent value and release anticipation.

Audio and video also introduce serious engineering challenges:

- much larger payload sizes,
- streaming and partial access concerns,
- compatibility with playback tooling,
- and efficient large-scale fragmentation.

That is why they are best described as the **next immediate targets after the first release**, not part of the current initial carrier set.

---

24. Long-Term Standardization Path

The correct path is:

1. publish the specification,
2. publish the whitepaper,
3. publish reference code,
4. publish test files and vectors,
5. invite review,
6. refine the standard,
7. and later pursue a formal standards discussion.

Datamorpho should mature through public scrutiny, not through hidden assumptions.

---

25. Conclusion

Datamorpho gives files latent states.

A Datamorphed file remains valid in its original format, publicly declares that hidden states exist, and still makes the recovery of those states depend on a separate state-specific secret-bearing reconstruction object.

Its strength lies in the disciplined combination of:

- public declaration,
- concealed payload,
- reconstruction secrecy,
- sparse or sparse-with-chaff layout,
- non-monotonic fragment placement,
- digest cross-binding,
- and cryptographic agility.

Datamorpho is not steganography.
It is not ordinary encryption.
It is not just a reveal trick.

It is a declared multi-state file standard.

That is why it deserves to exist.

Appendix A — Glossary

Base File
The original visible file used as the carrier input before Datamorpho embedding.

Carrier File
The final Datamorphed file.

Carrier Profile
The file-family-specific embedding rules, such as `jpeg-trailer`, `txt-envelope`, or `pdf-incremental`.

Chaff
Bytes intentionally present in a payload region but not used by the target hidden state, added to increase ambiguity and attack cost.

Concealed Payload
The payload bytes from which one or more hidden states can be reconstructed.

Datamorphed File
A file that conforms to a Datamorpho carrier profile and contains a public manifest plus concealed payload data.

Datamorphosis
The capability of a file to contain one or more hidden states beyond its visible representation.

Fragment
A span of bytes referenced by a reconstruction object as part of one hidden state.

Hidden State
A concealed alternate content state associated with a Datamorphed file.

Layout Strategy
The arrangement model for meaningful bytes in a payload region. In v0.001: `sparse` or `sparse-with-chaff`.

MorphoStorage
A locator or direction layer describing where to look for the reconstruction object of a state.

Public Manifest
The intentionally visible metadata declaring Datamorphosis, states, triggers, MorphoStorage, and reconstruction-object digests.

Reconstruction Object
A state-specific secret-bearing artifact containing the instructions and secret material needed to reconstruct one hidden state.

Sparse
A layout strategy in which meaningful bytes are non-contiguous and only referenced spans matter.

Sparse-with-Chaff
A layout strategy in which meaningful bytes are non-contiguous and unreferenced bytes are intentionally present to increase ambiguity.

Trigger
A declared condition associated with a hidden state, such as time, event, action, ownership, or custom logic.

Appendix B — Short FAQ

1. Is Datamorpho steganography?

No. Datamorpho requires a public declaration that hidden states exist. That is fundamentally different from a system designed to deny or conceal the existence of hidden content.

2. Does Datamorpho require one algorithm for the whole file?

No. A single hidden state may use different cryptographic suites across different fragments, and different states may use different combinations.

3. Does the reconstruction object contain the key?

In v0.001, yes. The reconstruction object is a complete secret-bearing artifact and may include the key material needed to reconstruct the target hidden state.

4. Does Datamorpho require states to be sequential?

No. States can be treated as an unordered set. They do not need to follow a linear release sequence.

5. Can bytes from different states coexist in the same payload region?

Yes. In Datamorpho, the same payload region may contain bytes relevant to different states, along with chaff.

6. Is Datamorpho limited to NFTs?

No. NFTs are only one important use case. Datamorpho is also relevant for games, publishing, archives, financial workflows, auctions, documents, news releases, and staged media.

7. Why allow `sparse` without chaff?

Because in some large-file or bandwidth-sensitive deployments, sparse non-contiguous layout may already be sufficient while avoiding payload growth and transfer overhead.

8. Does Datamorpho standardize viewers or trigger execution?

No. Datamorpho v0.001 standardizes the file structure and reconstruction semantics, not every possible viewer or trigger-validation mechanism.

9. Why start with JPEG, TXT, and PDF?

Because they give a strong initial set of carriers across media, text, and structured documents.

10. Why is the version 0.001 and not 0.002?

Because this is the first formal open specification release. The 2022 ETHGlobal Mexico 2022 project is the origin story and prototype lineage, not a prior published normative spec.

Appendix C — Release Positioning Blurb

Datamorpho is an open file standard for multi-state concealed content. A Datamorphed file remains valid in its original format while carrying one or more hidden states, publicly declaring their existence without disclosing the reconstruction logic needed to recover them. Its architecture combines cryptographic agility, sparse placement, optional chaff, non-monotonic fragment ordering, and state-specific

secret-bearing reconstruction objects to create a layered resistance model suitable for media, documents, games, collectibles, archives, financial workflows, and staged digital releases.